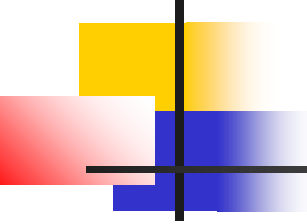


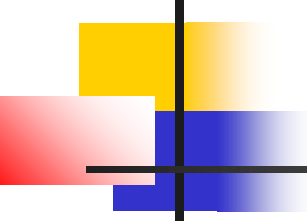
OPTIMAL TRAINING

Dr. Nguyen Thi Kim Ngan



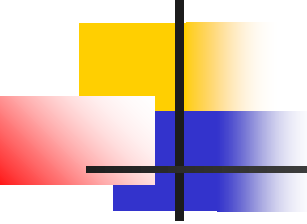
Outline

1. Gradient vanishing mitigation method
2. Training fine-tuning method
3. Transfer learning method



Outline

1. **Gradient vanishing mitigation method**
2. Training fine-tuning method
3. Transfer learning method



Gradient vanishing mitigation method

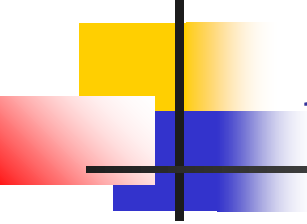
- For a deep neural network with L layers, input x , and output o . Each layer l is defined by a transformation f_l with parameters given by the weights W_l . This neural network can be represented as follows:

$$h^{l+1} = f_l(h^l), \text{ so } o = f_L \circ f_{L-1} \circ \dots \circ f_1(x)$$

- If all activations and inputs are vectors, we can rewrite the gradient of o with respect to an arbitrary parameter set W_l as follows:

$$\partial_{\mathbf{W}_l} \mathbf{o} = \underbrace{\partial_{\mathbf{h}^{L-1}} \mathbf{h}^L}_{:= \mathbf{M}_L} \cdot \dots \cdot \underbrace{\partial_{\mathbf{h}^l} \mathbf{h}^{l+1}}_{:= \mathbf{M}_l} \underbrace{\partial_{\mathbf{W}_l} \mathbf{h}^l}_{:= \mathbf{v}_l}.$$

- => If the gradient is multiplied by a very small derivative, learning becomes infeasible since the parameters hardly change at each update step (gradient vanishing)



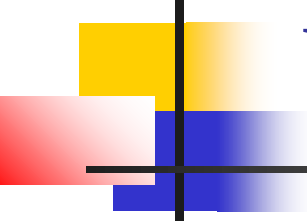
Activation Functions

- Sigmoid (problematic) $\sigma(x) = \frac{1}{1 + e^{-x}}, \quad \sigma'(x) = \sigma(x)(1 - \sigma(x))$

For large $|x|$, $\sigma'(x) \rightarrow 0$

- ReLU (Rectified Linear Unit): $f(x) = \max(0, x), \quad f'(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$
- Leaky ReLU $f(x) = \begin{cases} x & x > 0 \\ \alpha x & x \leq 0 \end{cases}, \quad f'(x) = \begin{cases} 1 & x > 0 \\ \alpha & x \leq 0 \end{cases}$

where $\alpha \in (0, 1)$



Weight initialization

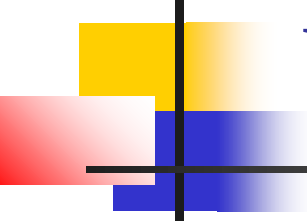
Good weight initialization can help mitigate the vanishing gradient problem

- Default initialization (problematic)

The weights W are sampled from a normal distribution $W \sim \mathcal{N}(0, 0.01^2)$
where mean $\mu=0$ and standard deviation $\sigma=0.01$

=>The initialized weights are **very small values** centered around 0

- Since the weights are tiny, forward signals and backpropagated gradients may **shrink too much** in deep networks → risk of **vanishing gradient**



Weight initialization

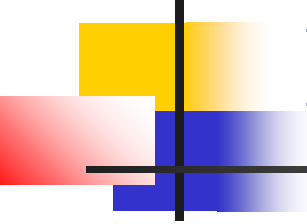
- Xavier/Glorot initialization

$$W \sim U \left[-\frac{\sqrt{6}}{\sqrt{n_{in} + n_{out}}}, \frac{\sqrt{6}}{\sqrt{n_{in} + n_{out}}} \right]$$

where n_{in} , n_{out} = number of input/output units

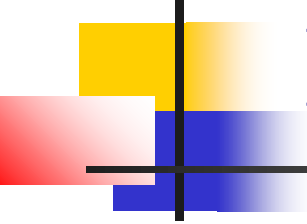
- Distributes weights **uniformly** within a range depending on input/output size
- Keeps the variance of activations approximately the same across layers
- Suitable for **sigmoid/tanh** activations

Result: prevents signals from shrinking to 0 or exploding when propagated forward/backward



Batch Normalization

- When training deep networks, the activations in the intermediate layers may exhibit large variations — across layers from input to output, across nodes within the same layer, and over time due to parameter updates
- Such changes in the distribution of activations can hinder the convergence of the network. It is evident that if the activations in one layer are 100 times larger than those in other layers, additional adjustments in the learning rate are required



Batch Normalization

Batch normalization is applied to individual layers (or possibly to all layers). The normalization is performed at each iteration, for each layer, as follows:

- At each layer: compute the activation values as usual
- Normalize the activation value of each neuron

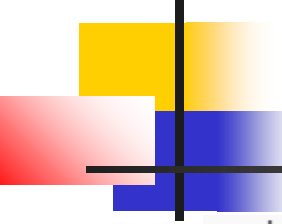
$$\text{BN}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \hat{\mu}}{\hat{\sigma}} + \beta$$

$\hat{\mu}$: mean value

$\hat{\sigma}$: standard deviation of the samples in the minibatch

γ : scaling factor

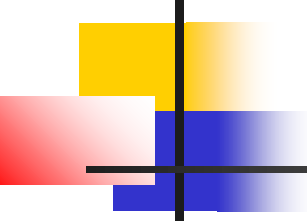
β : shifting (bias) parameter



Batch Normalization

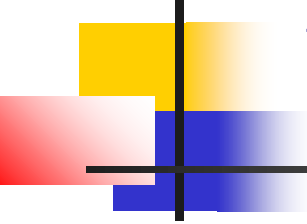
```
model = models.Sequential([  
    # First hidden layer (no activation yet)  
    layers.Dense(64, input_shape=(20,)), activation=None),  
    # Apply Batch Normalization to stabilize activations  
    layers.BatchNormalization(),  
    # Apply non-linear activation after BN  
    layers.Activation('relu'),  
  
    # Second hidden layer  
    layers.Dense(32, activation=None),  
    layers.BatchNormalization(),  
    layers.Activation('relu'),  
  
    # Output layer with softmax activation (classification into 2 classes)  
    layers.Dense(2, activation='softmax')  
])
```





Outline

1. Gradient vanishing mitigation method
2. Training fine-tuning method
3. Transfer learning method



Regularization

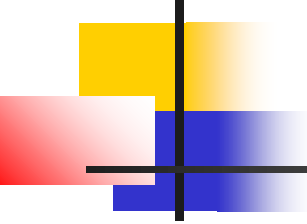
Add an additional term to the loss function.

- This term is often used to evaluate the complexity of the model
- The larger this term is, the more complex the model becomes

This new loss function is commonly referred to as a **regularized loss function**:

$$f_{\text{reg}}(w) = f(w) + \lambda R(w)$$

- w : parameters in the model
- $f(w)$: original loss function (depends on the training set and w)
- $R(w)$: regularization term (depends only on w)
- λ : regularization parameter, a small positive scalar to ensure the solution of $f_{\text{reg}}(w)$ is too far from the solution of $f(w)$

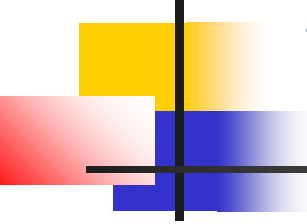


Regularization

Minimizing the *regularized loss function* is equivalent to minimizing both the *original loss function* and the *regularization term*

$$f_{\text{reg}}(w) = \underbrace{f(w)}_{\text{Tối thiểu}} + \lambda \underbrace{R(w)}_{\text{Tối thiểu}}$$

- LASSO regularization: $R(w) = \|w\|_1$
- RIDGE regularization: $R(w) = \|w\|_2^2$



Ridge regularization vs Linear Regression

Loss function of Linear Regression:

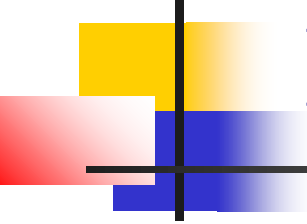
$$l(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} \left(\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)} \right)^2$$

Loss function with L2:

$$l(\mathbf{w}, b) + \frac{\lambda}{2} \|\mathbf{w}\|^2.$$

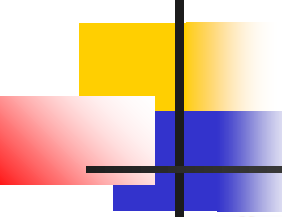
Stochastic Gradient update:

$$\mathbf{w} \leftarrow (1 - \eta\lambda) \mathbf{w} - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \mathbf{x}^{(i)} \left(\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)} \right)$$



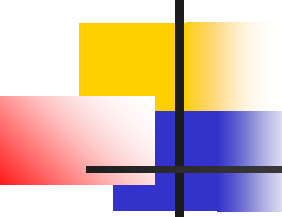
Dropout Technique

- Dropout is a regularization technique that randomly deactivates a subset of neurons during training to reduce overfitting
- At each training iteration, the dropout method sets the values of a certain proportion of nodes ($\leq 50\%$) in each layer to zero before computing the subsequent layers



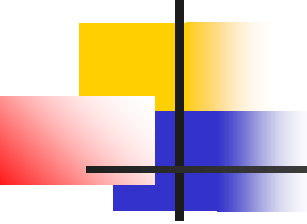
Dropout Technique

```
# dropout in the input layer with weight constraint
def create_model():
    # create model
    model = Sequential()
    model.add(Dropout(0.2, input_shape=(60,)))
    model.add(Dense(60, activation='relu', kernel_constraint=MaxNorm(3)))
    model.add(Dense(30, activation='relu', kernel_constraint=MaxNorm(3)))
    model.add(Dense(1, activation='sigmoid'))
    # Compile model
    sgd = SGD(learning_rate=0.1, momentum=0.9)
    model.compile(loss='binary_crossentropy', optimizer=sgd, metrics=['accuracy'])
    return model
```



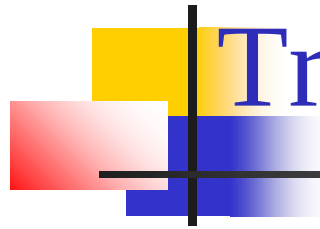
Dropout Technique

```
# dropout in hidden layers with weight constraint
def create_model():
    # create model
    model = Sequential()
    model.add(Dense(60, input_shape=(60,), activation='relu', kernel_constraint=MaxNorm(3)))
    model.add(Dropout(0.2))
    model.add(Dense(30, activation='relu', kernel_constraint=MaxNorm(3)))
    model.add(Dropout(0.2))
    model.add(Dense(1, activation='sigmoid'))
    # Compile model
    sgd = SGD(learning_rate=0.1, momentum=0.9)
    model.compile(loss='binary_crossentropy', optimizer=sgd, metrics=['accuracy'])
    return model
```



Outline

1. Gradient vanishing mitigation method
2. Training fine-tuning method
3. **Transfer learning method**



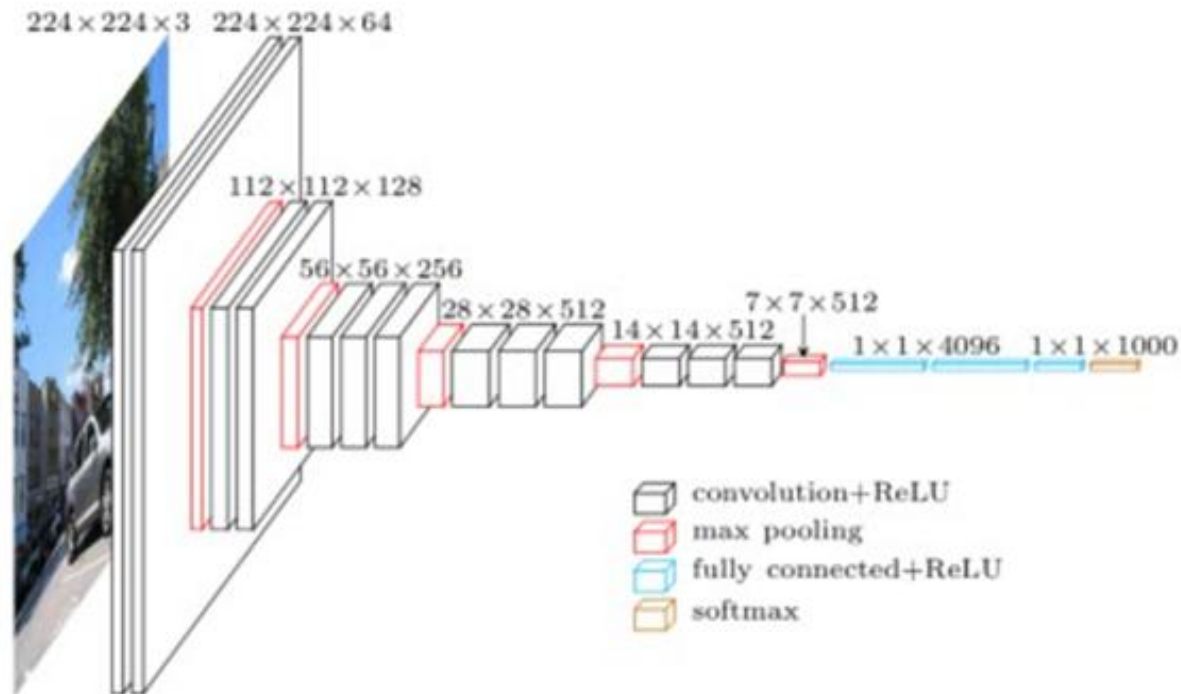
Transfer Learning

- The process of applying knowledge learned from a previous model (pretrained model) to the current task is called transfer learning
- Transfer learning leverages the features learned from pretrained models

Transfer Learning

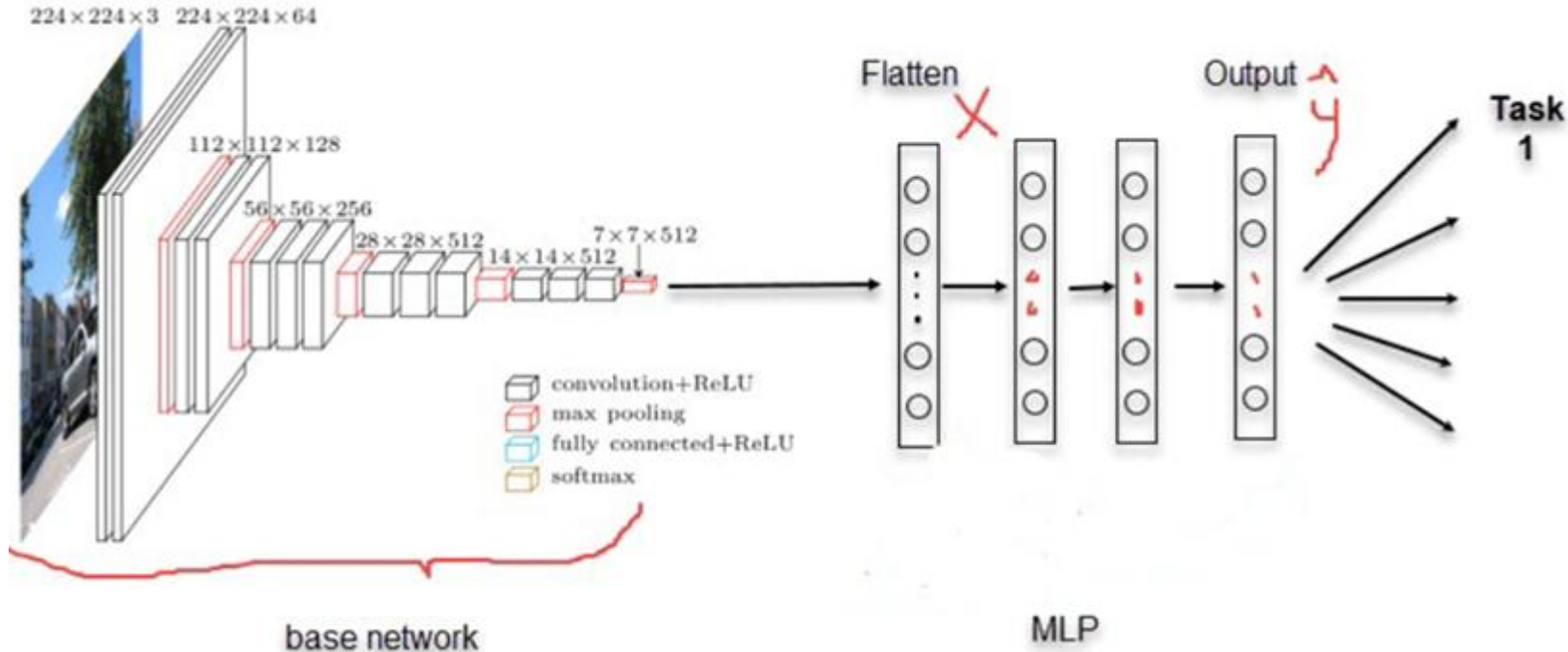
Phases of Transfer Learning:

- **Phrase 1:** A Base Network is a subnetwork extracted from a pretrained model after removing the top fully connected layers



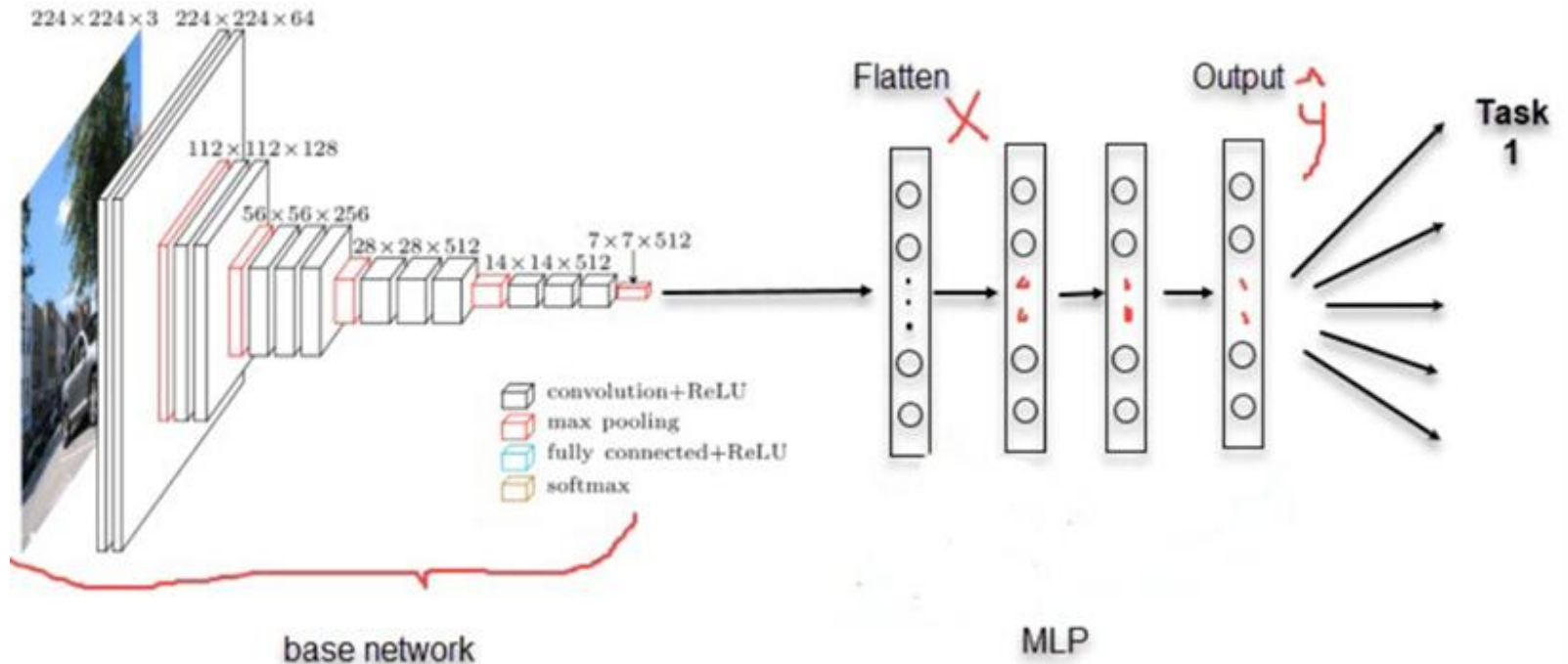
Transfer Learning

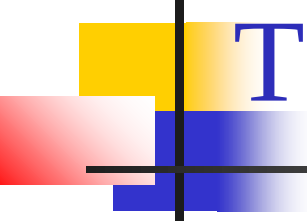
- **Phrase 2:** These are Fully Connected Layers that help reduce the dimensionality of data and compute the probability distribution at the output. Essentially, Fully Connected Layers are a form of a Multiple Layer Perceptron (MLP)



Transfer Learning

- Parameter Initialization:
 - Base Network: reuses the pretrained weights of the base network
 - MLP Network: the weights W and biases b of the MLP are initialized randomly
- The data passing through the base network produces meaningful features, which then serve as the input to the MLP network





Transfer Learning

Input Data



Base Network (pretrained weights)



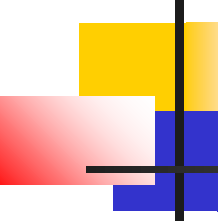
Extracted Features



MLP (randomly initialized W , b)

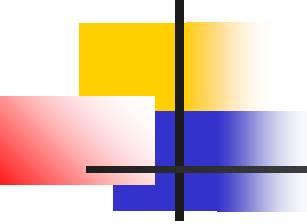


Output (Prediction)



Example

```
1 from tensorflow.keras.models import load_model, Sequential
2 from tensorflow.keras.layers import Dense, Flatten
3 from tensorflow.keras.applications import MobileNet
4 from tensorflow.keras.optimizers import Adam
5
6
7 base_network = MobileNet(input_shape=(224, 224, 3), include_top = False, weights = 'i
8 flat = Flatten()
9 den = Dense(1, activation='sigmoid')
10
11 model = Sequential([base_network,
12                     flat,
13                     den])
14 model.compile(optimizer=Adam(), loss='binary_crossentropy', metrics = ['accuracy'])
15 model.summary()
```

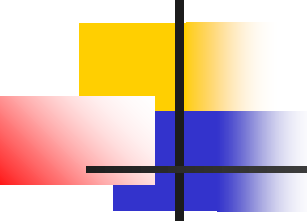


Gradient Descent

Let $f(\mathbf{x})$ be a continuously differentiable function, $\nabla f(\mathbf{x})$ its gradient, and learning rate $\eta > 0$

Original Gradient Descent Algorithm:

$$\mathbf{x} \leftarrow \mathbf{x} - \eta \nabla f(\mathbf{x})$$

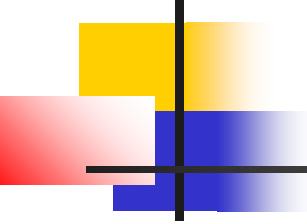


Gradient Descent

Let $f(\mathbf{x})$ be a continuously differentiable function, $\nabla f(\mathbf{x})$ its gradient, and learning rate $\eta > 0$

$H_f = \nabla \nabla^T f(\mathbf{x})$ is the Hessian matrix of f .

Thuật toán hạ gradi $\mathbf{x} \leftarrow \mathbf{x} - \eta \text{diag}(H_f)^{-1} \nabla f(\mathbf{x})$:



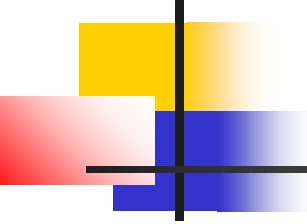
Gradient Descent

Assume the training set contains n samples, $f_i(x)$ is the loss function of the i -th sample, and $\mathbf{x}$ is the parameter vector. The objective function is given by:

$$f(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n f_i(\mathbf{x})$$

The gradient of the objective function at $\mathbf{x}$ is computed as follows:

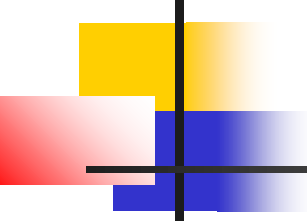
$$\nabla f(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\mathbf{x})$$



Gradient Descent

Stochastic Gradient Descent (SGD) reduces the computational cost per iteration. At each iteration, a data sample with index $i \in \{1, \dots, n\}$ is randomly selected uniformly, and the parameter $\mathbf{x}$ is updated using only the gradient $\nabla f_i(\mathbf{x})$

$$\nabla f(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\mathbf{x})$$



Gradient Descent

Minibatch Stochastic Gradient Descent. A minibatch $\mathcal{B}_t$ is a subset of the training dataset. Define:

$$\mathbf{g}_t = \partial_{\mathbf{w}} \frac{1}{|\mathcal{B}_t|} \sum_{i \in \mathcal{B}_t} f(\mathbf{x}_i, \mathbf{w})$$

Then: $\mathbf{w} = \mathbf{w} - \eta \mathbf{g}_t$